A Why a VAE has to exist

The goal we want, the bet we make, and the wall we slam into.

The goal

We want a model we can **sample new data from** — i.e. learn the true distribution P_x. Not just *recognise* data. **Create** it. Everything that follows exists to make that one thing trainable.

The bet: a hidden code

Real data is **thin** inside its own space. A 28×28 image is 784 numbers, but the set of plausible handwritten digits is a thin sheet inside $\mathbb{R}^{784}$. There is enormous redundancy.

So we **posit** that every **x** was produced from a short hidden code **z**:

$$z \in \mathbb{R}^k,\ k \ll d \quad \rightarrow \quad x \text{ (your data)}$$

CAREFUL — THIS IS A BET, NOT A FACT

We **invent** z. We never observe the real one — that is exactly why it is called **latent**. If the data has no such structure, the bet fails no matter how well we optimise.

The wall

Having a latent variable means our model has to **marginalise z out**:

$$P_\theta(x) = \int P_\theta(x, z)\, dz$$

An integral over **all of latent space**, sitting inside a log. There is no closed form, and you would need it at *every single* gradient step. So we cannot compute the likelihood — which kills maximum likelihood outright.

And it takes the posterior down with it, because the posterior needs that same quantity:

$$P_\theta(z|x) = P_\theta(x|z) \cdot P_\theta(z) \,/\, \mathbf{P_\theta(x)} \quad \leftarrow \text{the thing we just said we cannot compute}$$

WHY THIS IS A WALL AND NOT A SPEED BUMP

Training changes **θ** — the parameters. It does **not** change the fact that $P_\theta(x)$ is defined by an intractable integral. Swap in the perfect θ^* and the integral is still intractable. **You cannot train your way out of it.**

Sanity check that it really is about the *integral*: a Gaussian Mixture Model has the same philosophy, but its z is **discrete** — so the marginal is a finite **sum** of M terms, not an integral. That is why EM works perfectly there and collapses completely here.

Phase A in one line: we want to generate data → we bet on a hidden code → marginalising that code out is intractable → so the likelihood and the posterior are both out of reach.

B Bound it instead of computing it

What we do when the exact quantity is unreachable: squeeze it from below.

The move

We cannot compute log $P_\theta(x)$. So we stop trying, and instead build something we **can** compute that is guaranteed to sit **underneath** it.

Introduce **any** distribution q(z|x) — your own stand-in for the posterior. Multiply by 1 inside the integral, then apply **Jensen's inequality**:

$$\log P_\theta(x) \;\geq\; E_q\left[\log\left(P_\theta(x,z) \,/\, q(z|x)\right)\right] \;\equiv\; \textbf{ELBO}$$

That right-hand side is the **Evidence Lower Bound**. It contains only things we can evaluate: the model P_θ, and our own q.

But here is the sharper way to see it

The relationship is actually an **exact identity** — no inequality needed:

$$\log P_\theta(x) \;=\; \text{ELBO} \;+\; D_{KL}\left(q(z|x) \,\|\, P_\theta(z|x)\right)$$

$$\boxed{\log P_\theta(x)} \;=\; \boxed{\text{ELBO}} \;+\; \boxed{\text{gap} = D_{KL}(q \,\|\, \text{true posterior})}$$

the gap is **how wrong our q is** — it is what separates what we want from what we can compute.

Read it carefully, because it explains everything that follows:

- The ELBO is the log-likelihood **minus how wrong q is**.
- Since a KL is never negative, the ELBO is always a **lower bound**. ✓
- The bound is **tight** (gap = 0) **exactly when q equals the true posterior**.

THE TENSION THAT DEFINES THE WHOLE SUBJECT

We introduce q precisely to avoid the true posterior — but the bound is only good if q is close to the true posterior. We **want** $P_\theta(z|x)$; we **cannot have** it; so we approximate it and try to make the approximation as tight as we can.

WHY THAT GAP NEVER FULLY CLOSES

Our q can never *exactly* equal $P_\theta(z|x)$ — for two independent reasons:

- **q is confined to a family.** We only search over Gaussians whose μ and σ come out of a network. The true posterior need not be Gaussian, nor representable by that net at all.
- **The target moves.** $P_\theta(z|x)$ depends on θ, so every gradient step changes it — q is a function approximator chasing something that never stands still.

So the gap is narrowed, never closed. Note the two distinct layers: *computing* $P_\theta(z|x)$ at all is **structurally** impossible, whereas *matching* it is merely **imperfect**.

This is why it is called *variational* — we converted an intractable inference problem into an optimisation problem.

Phase B in one line: we can't compute the likelihood, so we compute a lower bound to it instead — and the distance between them is exactly how bad our approximation q is.

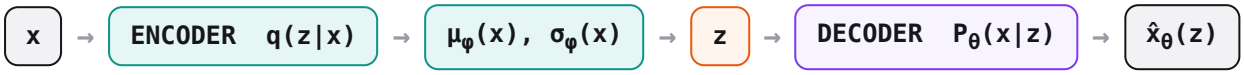
C Two networks fall out of the bound

Splitting the ELBO in two gives us an encoder and a decoder — and tells us which one is the real model.

Split the ELBO

$$\text{ELBO} = \underbrace{\mathbb{E}_{q(z|x)}[\log P_\theta(x|z)]}_{\text{reconstruction} \uparrow} - \underbrace{D_{KL}(q(z|x) \,\|\, P(z))}_{\text{regularisation} \uparrow}$$

Two objects appear, and each one is a network:



The critical detail — not the thing you'd expect

A neural network outputs **numbers**. It cannot output a distribution. So each network outputs the **parameters** of a distribution, and you supply the randomness separately:

ENCODER OUTPUTS

$\mu_\phi(x)$ and $\sigma_\phi(x)$ — the **mean and spread** of a Gaussian over z.

It does **not** hand you a z. It hands you a *blob*, and you sample from it.

DECODER OUTPUTS

$\hat{x}_\theta(z)$ — the **mean** of a Gaussian over x.

Again not a sample — the centre of a distribution. This becomes the reconstruction.

AND THE DETAIL THAT USUALLY GETS TAUGHT BACKWARDS

The two networks are **completely separate** — separate weights, no shared layers. They touch only through the sampled z handed between them. And crucially:

The decoder IS the generative model. It is what produces x. The encoder is **scaffolding** — a computational tool we invented to help us *train*, which is thrown away at generation time.

At generation you sample $z \sim N(0,I)$ and push it through the decoder. The encoder never runs.

Why an "auto"-encoder?

Because during training the pair is asked to squeeze x into z and then rebuild it — which looks like an autoencoder. But the mechanism is entirely different, and the KL term is what makes it something more than a compressor. (That's Phase E.)

Phase C in one line: the bound factors into "rebuild the input" and "keep the code well-behaved", giving a decoder (the real model) and an encoder (the training aid) — both of which output *parameters*, not samples.

D Sampling broke the gradient

A new problem, created by the fix in Phase C — and the one clever trick that resolves it.

The new problem

The encoder tells us where z lives. But to get an actual z we must **draw a random sample** from that distribution — and that draw sits **directly in the path** from the encoder's parameters to the loss:

$$z = \text{sample}(\, N(\mu_\phi(x),\ \sigma_\phi(x)^2)\,)$$

When the backward pass reaches that sampling step, it has **no derivative rule to apply**. It's an **opaque node** — a black box: parameters go in, randomness happens, a number comes out. The gradient chain is severed, and ϕ never learns.

The trick: move the randomness out of the parameters

Both of these produce **exactly the same distribution** for z:

× WHAT BLOCKS THE GRADIENT

`sample( μ, σ )` → `z`

The randomness is **inside** the thing whose parameters we must differentiate. Opaque → chain broken.

✓ WHAT LETS IT FLOW

`ε ~ N(0,1)` → `z = μ + σ·ε`

Randomness now lives in **ε**, a separate input. z is built from + and × — both differentiable.

Why are they the same? Because a Gaussian stays a Gaussian under shifting and scaling: if $\varepsilon \sim N(0,1)$, then $\mu + \sigma\cdot\varepsilon \sim N(\mu, \sigma^2)$. Same distribution, different *procedure*.

THE ONE DISTINCTION THAT MATTERS

ε is an input, not a parameter. Think of it exactly like the data x — something that *feeds* the network rather than something we tune.

- It receives **no gradient** (we never update ε).
- But it absolutely **shapes** the gradient to σ: since $z = \sigma\cdot\varepsilon$, we get $\partial J/\partial\sigma = (\partial J/\partial z)\cdot\varepsilon$. A bigger $|\varepsilon|$ means σ had a bigger lever on z, so it earns a bigger gradient.

No gradient flows *into* ε; but ε very much affects the gradient that flows *elsewhere*. Those are not contradictory.

What actually made the gradient computable was not ε itself — it was replacing an opaque `sample()` primitive with differentiable arithmetic. ε is merely the piece of that arithmetic that holds the randomness.

CONSEQUENCE WORTH CARRYING FORWARD

Not every distribution can be written as $z = g(\varepsilon, \phi)$ with ε from a fixed base. The ones that can are called **reparameterizable** — Gaussians, uniforms, exponentials. **That is why the encoder's q is a Gaussian.** It isn't just convenience; the trick demands it.

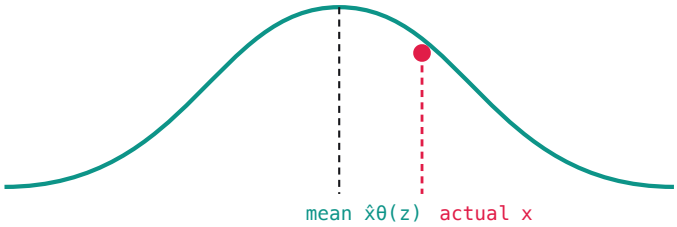
Phase D in one line: sampling from q_ϕ is opaque and kills the gradient, so we rewrite the same random z as $\mu + \sigma\cdot\varepsilon$ — moving the randomness into a fixed input and leaving a differentiable path behind.

E Turning the bound into a loss you can train

Both terms become ordinary, familiar quantities — and the reconstruction loss stops being a mystery.

Term 1 — where MSE actually comes from

Recall the decoder defines a Gaussian over x: $P_\theta(x|z) = N(x;\ \hat{x}_\theta(z),\ \sigma^2 I)$



A Gaussian peaks at its mean, so "make x probable" means **move the mean $\hat{x}$ onto x**:

$$\log P_\theta(x|z) = -(d/2)\log(2\pi\sigma^2) - \|x - \hat{x}_\theta(z)\|^2/(2\sigma^2) \quad \therefore \quad \text{maximising likelihood} \Leftrightarrow \textbf{minimising } \|x - \hat{x}_\theta(z)\|^2$$

THE REALISATION

The "reconstruction loss" is **not an invented design choice** — it is the log-likelihood of a Gaussian, rewritten. Change the likelihood (to Bernoulli, say, for binary pixels) and MSE becomes cross-entropy. **Choose the likelihood; the loss writes itself.**

(The dropped term $-(d/2)\log(2\pi\sigma^2)$ contains no x, no z and no θ — it is a constant, so removing it cannot move the optimum. Maximising f(θ)+5 is the same as maximising f(θ).)

Term 2 — the KL, in closed form

$$D_{KL}(\ q(z|x)\ \|\ N(0,I)\ ) = \tfrac{1}{2}\ \Sigma_j\ (\ \mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2\ ) \qquad \leftarrow \text{no sampling, no integral — just arithmetic on } \mu \text{ and } \sigma$$

And so the whole thing becomes a plain loss to minimise:

$$\textbf{LOSS} = \|x - \hat{x}_\theta(z)\|^2 + \tfrac{1}{2}\ \Sigma_j\ (\ \mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2\ )$$

What the two terms are actually fighting over

RECONSTRUCTION PULLS

"Put maximum information about x into z." Its cheapest win is to **memorise** — give every x its own isolated code, and drive σ → 0 so no noise blurs it.

KL PULLS BACK

"Keep q(z|x) near N(0,I)." It pushes σ up toward 1, turning each code into a **fuzzy blob** and forcing neighbouring blobs to **overlap**. Note the $-\log\sigma^2$ term explodes as σ → 0, so the KL actively *punishes certainty*.

WHY YOU MUST KEEP BOTH — AND WHY OVERLAP IS THE WHOLE POINT

A plain autoencoder (no KL) drives σ → 0, so codes become **isolated points** in mostly-empty space. Reconstruction is excellent — but sample a random z ~ N(0,I) and you land in a **gap**, and the decoder outputs garbage. **The KL term is what converts a compressor into a generator**: overlapping blobs tile the space, so *any* z lands somewhere meaningful.

Phase E in one line: the first term becomes MSE, the second a closed-form KL — and their tug-of-war is what makes the latent space both *informative* and *samplable*.